池州学院·考点对应模式 · 答案与解析

一、单项选择题（5×2=10分）

1.

答案：A

解析：按字节编址时，每个地址对应一个字节（1B）。14 根地址线可寻址 2^{14} = 16384 个地址，即 16384B = 16KB。A 正确。

- B 错误：将地址线根数"14"直接当作 KB 数，这是概念混淆，地址线根数与容量之间是指数关系（2^n），不是线性关系。
- C 错误：混淆了按字编址。若误认为按 16 位（2 字节）字编址，则 16384 / 2 = 8192 = 8KB，但题目明确说按字节编址。
- D 错误：混淆了数据线与地址线。若误将数据线位数（如 16 位 = 2 字节）乘入地址数：16384 × 2 = 32768 = 32KB，但按字节编址时每个地址仅 1 字节，不应再乘数据宽度。

2.

第一步：统计一帧总位数 N

一帧由起始位、数据位、校验位、停止位组成：

N = 1 + D + P + S = 1 + 8 + 1 + 2 = 12 （位）

第二步：代入有效数据传输率公式

有效数据传输率 = B × D / N：

有效数据传输率 = 9600 × 8/12 = 9600 × 2/3 = 6400 bps

正确答案：A. 6400 bps

选项	实际值	错误路径	说明
A	**6400 bps**	—（正确）	N=12，9600×8/12 = 6400
B	约 6982 bps	忽略校验位	帧总位数少算 1 位，N'=1+8+0+2=11，9600×8/11 ≈ 6981.82
C	7680 bps	忽略停止位	帧总位数少算 2 位，N''=1+8+1+0=10，9600×8/10 = 7680
D	9600 bps	直接用波特率	将波特率直接当作有效数据传输率

各选项互异验证：6400、6982、7680、9600 四个值各不相同。

3.

答案：C

解析：中断分为外部中断（硬件中断/外部事件引起）和内部中断（异常/程序执行过程中由指令本身引起）。C 正确：定时器（时钟）是外部硬件设备，其计数到期向 CPU 发出的中断请求属于典型的外部中断。

- A 错误：除数为 0 是 CPU 在执行除法指令时检测到的程序性错误，属于内部中断（异常）。
- B 错误：地址越界是 CPU 访存时检测到的违例，属于内部中断（异常）。
- D 错误：缺页是在指令执行过程中由存储管理部件检测到的异常，属于内部中断。

4.

答案：C

解析：存储层次从靠近 CPU 到远离 CPU 的顺序为：寄存器 → Cache → 主存 → 辅存，速度递减、容量递增、每位价格递减。C 正确：寄存器位于 CPU 内部，存取速度最快，但容量最小（通常只有几十个字）。

- A 错误：Cache 的存取速度高于主存（否则 Cache 失去意义），容量也小于主存。
- B 错误：主存的存取速度低于 Cache，容量大于 Cache。
- D 错误：辅存的存取速度远低于主存（辅存是磁盘/固态盘等），虽然容量确实最大，但速度判断错误。

5.

答案：C

解析：立即寻址的操作数直接包含在指令中，取指令的同时即获得操作数，无需额外访问内存或寄存器，因此取操作数速度最快。C 正确。

- A 错误：直接寻址需要根据指令中的形式地址访问一次内存才能获得操作数，比立即寻址慢。
- B 错误：间接寻址需要先访问内存取得有效地址，再访问内存取得操作数，至少两次访存，是最慢的。
- D 错误：寄存器间接寻址需要先从寄存器取得地址，再访问内存取得操作数，虽然比间接寻址快，但仍需访存，不如立即寻址快。

二、判断题（5×2=10分）

1.

答案：① √　② √　③ ×　④ √

解析：错误。算术运算与逻辑运算是**运算器（ALU）**的职责；控制器负责解释指令、产生控制信号、协调各部件，并不执行运算——此处把运算器的功能安到了控制器上。

2.

答案：① √　② √　③ ×　④ ×

解析：正确。异步通信不依赖公共时钟，而由握手信号（请求/回答）实现主从模块的时序协调。

3.

答案：① √　② √　③ √　④ ×

解析：正确。存储层次两级目标不同：Cache—主存基于局部性原理提速，主存—辅存借助虚拟存储扩容。

4.

答案：① √　② ×　③ √　④ √

解析：错误。水平型微指令含多位控制字段、可并行定义多个微操作，字长更长；垂直型微指令类似机器指令（操作码+地址码），字长较短——此处正相反。

5.

答案：① √　② ×　③ ×　④ √

解析：错误。指令周期与指令功能及寻址方式有关：乘法通常需多次"加法+移位"，机器周期数多于普通加法，故二者指令周期不同。

三、应用题（60分）

1.

补码加减运算与溢出判断 — 解答

基本参数

参数	值
数据位 n	3
符号位	2（双符号位）
总位数	5
模 M	$2^5 = 32$
有效表示范围	$[-2^3, 2^3-1] = [-8, 7]$

第 1 问：求 A、B 的变形补码（5位）

A = -5 的变形补码

1. 取绝对值：$|A| = 5 = 101_{(2)}$（3 位）
2. 逐位取反：101 → 010
3. 末位加一：010 + 1 = 011
4. 符号扩展为双符号位：前面补 11

$[A]_{变补} = \boxed{11011}$

B = -6 的变形补码

1. 取绝对值：$|B| = 6 = 110_{(2)}$（3 位）
2. 逐位取反：110 → 001
3. 末位加一：001 + 1 = 010
4. 符号扩展为双符号位：前面补 11

$[B]_{变补} = \boxed{11010}$

第 2 问：计算 A + B

利用第 1 问的变形补码，逐位相加：

```
    1 1 0 1 1    ([A]变补 = 27)
+   1 1 0 1 0    ([B]变补 = 26)
──────────────
  1 1 0 1 0 1    (原始和 = 53，7位)
```

原始和为 $53 = 110101_{(2)}$（7 位），丢弃最高位进位后取低 5 位：

$[A+B]_{变补} = 10101 \ (= 21)$

双符号位判断：结果的双符号位 $S_1\ S_0 = 10$。

- $S_1 = 1, S_0 = 0$，$S_1 \neq S_0$ → **溢出**
- $S_1 = 1, S_0 = 0$ → **负溢出（下溢）**

验证：A + B = -5 + (-6) = -11，真值 -11 超出有效表示范围 [-8, 7]，确实溢出。双符号位判溢结论与真值范围一致。✓

$\boxed{[A+B]_{变补} = 10101，\ 双符号位\ 10，\ \textbf{负溢出（下溢）}}$

第 3 问：计算 A - B

第一步：将减法转化为加法

A - B = A + (-B)，需要对 $[B]_{变补}$ 连同符号位取反加一，得到 $[-B]_{变补}$。

1. $[B]_{变补}$ = 11010
2. 连同符号位取反：11010 → 00101
3. 末位加一：00101 + 1 = 00110

$[-B]_{变补}$ = 00110 \ (= 6)

> 验证：-B = -(-6) = 6，6 的变形补码确为 00110。取反加一路径与直接求 (-B) 的变形补码一致。√

第二步：逐位相加

```
    1 1 0 1 1    ([A]变补 = 27)
+   0 0 1 1 0    ([-B]变补 = 6)
─────────────
  1 0 0 0 0 1    (原始和 = 33，6位)
```

原始和为 33 = $100001_{(2)}$（6 位），丢弃最高位进位后取低 5 位：

$[A-B]_{变补}$ = 00001 \ (= 1)

第三步：溢出判断与真值还原

双符号位判断：结果的双符号位 $S_1\ S_0$ = 00。

- S_1 = 0,\ S_0 = 0，$S_1 = S_0$ → **未溢出**
- 双符号位 00 → 结果为正

真值还原：双符号位 00（正数，未溢出），真值 = $00001_{(2)}$ = 1。

验证：A - B = -5 - (-6) = -5 + 6 = 1，真值 1 在有效表示范围 [-8, 7] 内，确实未溢出。还原真值 1 与实际真值 1 一致。√

\boxed{$[A-B]_{变补}$ = 00001，\ 双符号位 00，\ \textbf{未溢出}，\ 真值 = 1}

场景规则验证汇总

规则	内容	结果
规则 1	A=-5 ∈ [-8,7]，B=-6 ∈ [-8,7]	√

规则	内容	结果
规则 2	A+B=-11 溢出，A-B=1 未溢出，至少一组溢出	√
规则 3	$[-B]_{变补}$ 取反加一路径（00110）= 直接求补（00110）；未溢出时还原真值 1 = A-B	√
规则 4	A+B：双符号位判溢（溢出）= 真值超出范围（True）；A-B：双符号位判溢（未溢出）= 真值超出范围（False）	√

2.

问题	答案
第1问 指误字	$S = S_4S_2S_1$ = **101**（十进制 5）
第2问 错误位	**H_5**，纠错后码字 = **0110011**
第3问 原始数据	$D_1D_2D_3D_4$ = **1011**

3.

（1）地址线数与数据线数

目标字数（容量自洽 R1）：

每字字节数 = B_target / 8 = 16 / 8 = **2 字节**

目标字数 = C_target / 每字字节数 = 65536 / 2 = **32768 = 32K 字**

整除验证（R2）：16 % 8 = 0，32768 % 8192 = 0，均整除，参数合法。

地址线数（R3）：按字编址，每字对应一个地址，地址线数 = $\log_2$(目标字数) = $\log_2$(32768) = **15 根**

数据线数（R4）：数据线数 = B_target = **16 根**

> ✅ RULE_CHECK R3: $\log_2$(32768) = 15，2^15 = 32768 √
> ✅ RULE_CHECK R4: B_target = 16 == 数据线 16 √

答（1）：地址线 15 根，数据线 16 根。

（2）芯片组数（字扩展）与每组芯片数（位扩展）

芯片组数 G（字扩展方向）（R5）：

G = 目标字数 / 芯片字数 = 32768 / 8192 = **4**

> ✅ RULE_CHECK R5: G = 4 ⩾ 2 √

每组芯片数 P（位扩展方向）（R6）：

P = B_target / B_chip = 16 / 8 = **2**

> ✅ RULE_CHECK R6: P = 2 ⩾ 2 √

答（2）：需要 4 个芯片组（字扩展），每组需要 2 片芯片（位扩展）。

（3）总芯片数

总芯片数（R7）：

总芯片数 = G × P = 4 × 2 = **8 片**

总量校验（R8）： - 字扩展覆盖：G × W_chip = 4 × 8192 = 32768 == 目标字数 32768 √ - 位扩展覆盖：P × B_chip = 2 × 8 = 16 == B_target 16 √

答（3）：总共需要 8 片 8K × 8 位的 SRAM 芯片。

子问	结论
（1）	地址线 = 15 根，数据线 = 16 根
（2）	芯片组数 G = 4（字扩展），每组芯片数 P = 2（位扩展）
（3）	总芯片数 = G × P = 4 × 2 = 8 片

4.

问题	答案
Q1(a) 一地址操作码位数	**12 位**
Q1(b) 一地址指令最大条数	**3999 条**
Q1(c) 至少保留一地址前缀数	**1 个**
Q2 零地址指令最大条数	**16 条**

5.

子问题	答案
1. L0 屏蔽字	**11111**
1. L1 屏蔽字	**01101**

子问题	答案
1. L2 屏蔽字	**00100**
1. L3 屏蔽字	**01111**
1. L4 屏蔽字	**00101**
2(a)	L3 **能**打断 L1（L1 屏蔽字第 L3 位 = 0，处理优先级 L3 > L1）
2(b)	L2 **不能**打断 L3（L3 屏蔽字第 L2 位 = 1，处理优先级 L2 < L3）
2(c) 服务开始顺序	**L1 → L3**
2(c) 服务完成顺序	**L3 → L1 → L2**

6.

解答

第(1)问（2分）

补码小数表示

- x = +0.5：正数，补码 = 原码 = 0.1000
- y = -0.625：负数，|y| = 0.625 = 0.1010（原码），补码 = 原码数值位逐位取反末位加 1：取反得 1.0101，加 1 得 1.0110
- [-x]_补：[x]_补 = 0.1000，逐位取反得 1.0111，末位加 1 得 1.1000

> 验证：[x]_补 + [-x]_补 = 0.1000 + 1.1000 = 0.0000（模 2 为 0），正确。

初始状态

寄存器	初始值
部分积 PP（6位双符号位）	00.0000
乘数 MQ（5位）	10110
附加位	0

第(2)问（6分）

Booth 算法逐趟推演

初始：PP = 00.0000，MQ = 10110，附加位 = 0

第1趟

项目	值
当前 PP / MQ / 附加位	00.0000 / 10110 / 0
判断位（MQ最低位 + 附加位）	0, 0 \Rightarrow 00
操作	+0
加减后 PP	00.0000
右移后 PP / MQ / 附加位	00.0000 / 01011 / 0

第2趟

项目	值
当前 PP / MQ / 附加位	00.0000 / 01011 / 0
判断位（MQ最低位 + 附加位）	1, 0 \Rightarrow 10
操作	+[-x]_补 = +11.1000
加减后 PP	11.1000
右移后 PP / MQ / 附加位	11.1100 / 00101 / 1

> 算术右移：PP 双符号位最高位为 1，符号扩展，11.1000 → 11.1100。

第3趟

项目	值
当前 PP / MQ / 附加位	11.1100 / 00101 / 1
判断位（MQ最低位 + 附加位）	1, 1 \Rightarrow 11
操作	+0
加减后 PP	11.1100
右移后 PP / MQ / 附加位	11.1110 / 00010 / 1

> 符号扩展：11.1100 → 11.1110。

第4趟

项目	值
当前 PP / MQ / 附加位	11.1110 / 00010 / 1
判断位（MQ最低位 + 附加位）	0, 1 \Rightarrow 01
操作	+[x]_补 = +00.1000
加减后 PP	00.0110
右移后 PP / MQ / 附加位	00.0011 / 00001 / 0

> 11.1110 + 00.1000 = 100.0110，双符号位取低 6 位 = 00.0110。符号扩展：00.0110 → 00.0011。

第5趟

项目	值
当前 PP / MQ / 附加位	00.0011 / 00001 / 0
判断位（MQ最低位 + 附加位）	1, 0 \Rightarrow 10
操作	+[-x]_补 = +11.1000
加减后 PP	11.1011
右移后 PP / MQ / 附加位	11.1101 / 10000 / 1

> 第 5 趟处理乘数符号位。乘数 y = 1.0110 为负数，符号位（最高位）为 1，与前一数值位 0 构成判断位 10，需执行 +[-x]_补。00.0011 + 11.1000 = 11.1011。符号扩展：11.1011 → 11.1101。

最终乘积

最终 PP = 11.1101，MQ = 10000。

取双符号位 PP 的低 5 位（去掉冗余的最高符号位）与 MQ 拼接，得到 **10 位乘积补码表示**：

\boxed{乘积 = 1.110110000 \quad (即 1110110000_2)}

乘积真值：10 位乘积最高位为冗余符号位，有效位为低 9 位 110110000。按 9 位补码小数解码：

-1 + 1/2 + 0 × 1/4 + 1/8 + 1/16 + 0 + 0 + 0 + 0 = -256/256 + 128/256 + 32/256 + 16/256 = -80/256 = -5/16

> 验证：x × y = 1/2 × \left(-5/8\right) = -5/16 = -0.3125

\boxed{乘积真值 = -5/16 = -0.3125}

场景规则验证

规则	结果	说明
Rule1 补码有效性	√	[x]_补=0.1000 解码 =+0.5，[y]_补=1.0110 解码 =-0.625
Rule2 [-x]_补 正确性	√	逐位取反末位加 1 = 1.1000；[x]_补+[-x]_补 \bmod 2 = 0
Rule3 位模式覆盖	√	[y]_补=10110 同时含 "01" 和 "10" 模式
Rule4 乘积正确性	√	-5/16 = x × y
Rule5 双符号位一致	√	每趟 PP 保持 6 位双符号位，算术右移正确符号扩展
Rule6 符号位参与运算	√	第 5 趟判断位 "10"，正确执行 +[-x]_补 后移位
Rule7 无溢出	√	乘积真值 -5/16 ∈ [-1, +1)

7.

ROM 区域：

- ROM 结束地址 = 起始地址 + 容量 − 1 = 0000H + 4096 − 1 = **0FFFH**
- ROM 地址范围 = **0000H ~ 0FFFH**

RAM 区域（紧接 ROM 之后）：

- RAM 起始地址 = ROM 结束地址 + 1 = 0FFFH + 1 = **1000H**
- RAM 结束地址 = RAM 起始地址 + 容量 − 1 = 1000H + 8192 − 1 = **2FFFH**
- RAM 地址范围 = **1000H ~ 2FFFH**

> 场景规则验证（R1/R2）：RAM 起始 = ROM 结束 + 1 √；ROM 结束（0FFFH）< RAM 起始（1000H），不重叠 √；ROM + RAM = 4KB + 8KB = 12KB ⩽ 64KB √。

Q3.1 译码器输出端地址区间与连接

片内寻址位数（计算）= $\max(\log_2(2048), \log_2(2048))$ = **11 位**

译码粒度 = 2^{11} = **2048 B = 2 KB**

CBA 连接：**C = A13, B = A12, A = A11**（即 CBA 连接的地址线编号 = 片内寻址位数 + {2, 1, 0} = 11+{2,1,0} = {13,12,11}）

> 场景规则验证（R5）：片内寻址位数 = 11 √；译码粒度 = 2KB √；C=A13=A11+2, B=A12=A11+1 √。

各译码输出端对应 2 KB 地址区间：

输出端	地址范围	CBA(A13,A12,A11)	连接
Y0	0000H ~ 07FFH	0 0 0	ROM
Y1	0800H ~ 0FFFH	0 0 1	ROM
Y2	1000H ~ 17FFH	0 1 0	RAM
Y3	1800H ~ 1FFFH	0 1 1	RAM
Y4	2000H ~ 27FFH	1 0 0	RAM
Y5	2800H ~ 2FFFH	1 0 1	RAM
Y6	3000H ~ 37FFH	1 1 0	未用
Y7	3800H ~ 3FFFH	1 1 1	未用

- ROM 占用 **2 块**（Y0~Y1），RAM 占用 **4 块**（Y2~Y5），总 **6 块**。
- ROM 用输出端：**Y0、Y1**
- RAM 用输出端：**Y2、Y3、Y4、Y5**

> 场景规则验证（R6）：总块数 6 ⩽ 8（74LS138 输出端数）√。

Q3.2 R/W 信号需求差异

- **ROM 区域**：不需要 R/W 信号参与写控制——ROM 只读，片选信号有效即可输出数据。
- **RAM 区域**：需要 R/W 信号——RAM 可读可写，R/W 决定数据方向（高 = 读，低 = 写）。

Q3.3 使能端连接方案（部分译码）

使能端	连接	说明
G1（高有效）	接 Vcc（+5V 恒高）	恒为有效
G2A（低有效）	接 MREQ（低有效）	访存时 MREQ=0 使能
G2B（低有效）	接 A15（低有效）	A15=0 使译码器在低 32KB（0000H~7FFFH）工作

(1) 译码器实际工作地址范围：

译码器工作条件 = (Vcc = 1) **且** (MREQ = 0) **且** (A15 = 0)。即 CPU 发出访存请求且目标地址 A15 = 0（地址落在 0000H~7FFFH 的低 32KB）时，译码器才工作。注意 G2B 只接 A15，限定的是**低 32KB 半区**，而非仅有的 12KB 存储区。

(2) 地址镜像（部分译码重叠）：

本题译码输入 CBA = A13/A12/A11（区分 Y0~Y7 对应 0000H~3FFFH 的 16KB），**A14 既未参与译码也未参与使能**。因此当 A14 = 1（即 4000H~7FFFH）时，CBA 的取值与 A14 = 0 时完全相同，译码器同样会选中 Y0~Y5，使存储区被"复制"一份：

- 原存储区：ROM 0000H~0FFFH、RAM 1000H~2FFFH（共 12KB，A14 = 0）。
- 镜像区间：**4000H~6FFFH**（共 12KB，A15 = 0、A14 = 1）——例如 4000H 与 0000H 选中同一片 ROM。

这是典型的**部分译码（partial decode）**产生的地址重叠。

(3) 部分译码在本题是否可接受：

可接受。本题存储区仅占 12KB，镜像区 4000H~6FFFH 未与其他外设/存储器分配冲突，且部分译码省去了对 A14 的额外译码逻辑、简化硬件。若要求全译码（消除镜像），需把 A14 也纳入使能（如 G2B = A15 + A14 之类的逻辑），本题未作此要求。

> 场景规则验证（R7/R8）： - R7：使能端逻辑——存储区内地址 0000H：A15=0 → G2B 有效（译码器工作）√；8000H：A15=1 → G2B 无效（译码器禁止）√。工作条件 = (MREQ == 0) 且 (A15 == 0) 且 (Vcc == 1) √。 - R8：ROM 结束 0FFFH、RAM 结束 2FFFH 的 A15 均为 0 √，存储区落在 G2B=A15 使能的低 32KB 内（注：A14 未参与，存在 4000H~6FFFH 镜像，见上文）。

8.

Cache 地址映射格式设计 — 参考解答

基本参数（PARAMS）

参数	值
主存容量 M	2^{28} B = 268 435 456 B
Cache 容量 C	2^{15} B = 32 768 B
块长 B	2^{5} B = 32 B
组相联路数	4（四路组相联）

由 PARAMS 推导出的关键中间值（PROCESS）：

量	计算	值
主存地址位数	$\log_2(M) = \log_2(2^{28})$	**28**
块内地址位数	$\log_2(B) = \log_2(2^{5})$	**5**

量	计算	值
Cache 行数	$C/B = 2^{15} / 2^5 = 2^{10}$	**1024**
组数（四路）	$C/(B\times4) = 2^{15} / (2^5 \times 4) = 2^{15} / 2^7 = 2^8$	**256**

第（1）题（2 分）— 直接映射

Cache 行数 = C/B = 1024 行，行号位 = $\log_2(1024)$ = **10** 位。

块内地址位 = $\log_2(B) = \log_2(32)$ = **5** 位。

标记位 = 28 − 10 − 5 = **13** 位。

主存地址总长度 = 13 + 10 + 5 = **28 位**。

字段	标记	行号（Cache 行号）	块内地址
位数	13	10	5

> **地址格式**（高位 → 低位）： `标记 13 位 | 行号 10 位 | 块内地址 5 位`

第（2）题（3 分）— 四路组相联映射

① Cache 共有多少组？组号需要多少位？

- Cache 行数 = C/B = 1024 行。
- 每组包含 4 个 Cache 行（四路），故组数 = 1024 / 4 = **256 组**。
- 组号位 = $\log_2(256)$ = **8 位**。

② 主存地址各字段

组号位 = $\log_2(256)$ = **8** 位。

块内地址位 = $\log_2(B)$ = **5** 位。

标记位 = 28 − 8 − 5 = **15** 位。

字段	标记	组号	块内地址
位数	15	8	5

> **地址格式**（高位 → 低位）： `标记 15 位 | 组号 8 位 | 块内地址 5 位`

第（3）题（2 分）— 全相联映射

全相联映射无索引字段（不区分行号/组号）。

块内地址位 = $\log_2(B)$ = **5** 位。

标记位 = 28 − 5 = **23** 位。

字段	标记	块内地址
位数	23	5

> **地址格式**（高位 → 低位）： `标记 23 位 | 块内地址 5 位`

第（4）题（1 分）— 格式汇总

映射方式	标记位数	索引（行号/组号）位数	块内地址位数	地址总位数
直接映射	**13**	**10**	**5**	**28**
四路组相联	**15**	**8**	**5**	**28**
全相联	**23**	**0**（无）	**5**	**28**

验证（RULE_CHECK）

三种映射方式均满足**地址位数守恒**（标记 + 索引 + 块内 = 28）：

- 直接映射：13 + 10 + 5 = 28 √
- 四路组相联：15 + 8 + 5 = 28 √
- 全相联：23 + 0 + 5 = 28 √

其他场景规则均通过（参数为 2 的幂、M > C > B 且 C/B = 1024 ⩾ 4、各字段位数 ⩾ 1、块内地址位数 5 ∈ [4, 8]、标记位均 ⩾ 4），详见 gen.py RULE_CHECK 输出。

四、综合题（2×10=20分）

1.

(1) 直接寻址（MOD = 0001）

直接寻址：EA = A，操作数 = M[EA]。

- EA = A = 0x0040
- 操作数 = M[0x0040] = 0x0048

> **有效地址 EA = 0x0040，操作数 = 0x0048**

访存次数：1 次。

---

(2) 间接寻址（MOD = 0010）

间接寻址：EA = M[A]（先访存读出指针），操作数 = M[EA]（再访存取操作数）。

- 第 1 次访存：读出指针 = M[0x0040] = 0x0048
- EA = 0x0048
- 第 2 次访存：操作数 = M[0x0048] = 0x1234

> **有效地址 EA = 0x0048，操作数 = 0x1234**

访存次数：2 次（间接寻址至少需 2 次访存）。

---

(3) 相对寻址（MOD = 0011）

相对寻址：EA = PC_updated + A，其中 PC_updated = 当前指令首地址 + 指令占用字地址数 = 0x000E + 2 = 0x0010。

- PC_updated = 0x0010（取指后 PC 指向下一条指令，**不是**当前指令地址 0x000E）
- EA = 0x0010 + 0x0040 = 0x0050
- 操作数 = M[0x0050] = 0x5678

> **有效地址 EA = 0x0050，操作数 = 0x5678**

访存次数：1 次。

陷阱说明：若误用当前指令地址 0x000E 而非取指后值 0x0010，则错误 EA = 0x000E + 0x0040 = 0x004E，操作数 = M[0x004E] = 0xABCD。该错误结果看似合理（0x004E 在主存表中），但与正确 EA 0x0050 不同，是典型陷阱。

---

(4) 变址寻址（MOD = 0100）

变址寻址：EA = Rx + A，操作数 = M[EA]。

- EA = Rx + A = 0x0018 + 0x0040 = 0x0058
- 操作数 = M[0x0058] = 0x9ABC

> **有效地址 EA = 0x0058，操作数 = 0x9ABC**

访存次数：1 次。

---

(5) 基址寻址（MOD = 0101）

基址寻址：EA = Rb + A，操作数 = M[EA]。

- EA = Rb + A = 0x0020 + 0x0040 = 0x0060
- 操作数 = M[0x0060] = 0xDEF0

> **有效地址 EA = 0x0060，操作数 = 0xDEF0**

访存次数：1 次。

小问	寻址方式	有效地址 EA	操作数	访存次数
(1)	直接寻址	0x0040	0x0048	1
(2)	间接寻址	0x0048	0x1234	2
(3)	相对寻址	0x0050	0x5678	1
(4)	变址寻址	0x0058	0x9ABC	1
(5)	基址寻址	0x0060	0xDEF0	1

2.

指令流水线性能与冒险分析 — 参考解答

> 以下解答数据均由 gen.py exec 产出（逐字对照 gen.py stdout 转写，不重算）。

基本参数

参数	值
流水线级数 n	5（IF / ID / EX / MEM / WB）
各级时间	IF=300 ps, ID=200 ps, EX=500 ps, MEM=400 ps, WB=200 ps
指令条数 L	4
各级时间之和	300+200+500+400+200 = 1600 ps

第（1）题（2分）：时钟周期

流水线各级在同一时钟驱动下工作，时钟周期取各段时间的最大值：

$T_{clk} = \max(300, 200, 500, 400, 200) = 500\ ps$

依据：流水线各段固定在同一时钟节拍下运行，时钟周期必须能容纳最慢的那一级（此处为 EX = 500 ps）。

第（2）题（3分）：理想流水线性能

（a）理想流水线总时间

$T_{ideal} = (n + L - 1) \times T_{clk} = (5+4-1) \times 500 = 8 \times 500 = 4000\ ps$

（b）加速比 S

非流水线执行时间（每条指令串行走完全部 5 级）：

$T_{non} = L \times (IF+ID+EX+MEM+WB) = 4 \times 1600 = 6400\ ps$

$S = \frac{T_{non}}{T_{ideal}} = 6400/4000 = 1.6$

（S = 1.6 > 1 且非整数，符合预期。）

（c）吞吐率

$TP = \frac{L}{T_{ideal}} = 4/4000\ ps = 0.001\ 条/ps = \mathbf{1.00\ 指令/ns}$

第（3）题（2分）：数据冒险分析

逐对数据相关

相关对	相关寄存器	类型
I1（LW）→ I2（ADD）	R1	**Load-use 相关**（producer 为 Load 指令）
I2（ADD）→ I3（SUB）	R3	一般 RAW 相关（ALU→ALU）
I3（SUB）→ I4（SW）	R5	一般 RAW 相关（ALU→ALU）

说明： - I1 写 R1，I2 读 R1 → RAW 相关，且 I1 是 Load → Load-use 相关。 - I2 写 R3，I3 读 R3 → RAW 相关（ALU→ALU）。 - I3 写 R5，I4 读 R5（Store 的源数据） → RAW 相关（ALU→ALU）。

WAR / WAW 冒险

在**按序（in-order）流水线**中，指令严格按程序顺序进入、按序完成，**不存在 WAR 和 WAW 冒险**： - **WAR**（Write After Read）要求后续指令先于前序指令写寄存器，按序流水线不会发生这种乱序。 - **WAW**（Write After Write）要求后续指令先于前序指令写同一寄存器，同理按序流水线不会发生。

本序列也无逆序的写-读或写-写对，因此 **WAR 相关对数 = 0，WAW 相关对数 = 0**。

第（4）题（3分）：停顿周期计算

理想流水线周期数

理想周期 = n + L - 1 = 5 + 4 - 1 = 8\ 周期

（a）无转发机制

无转发时，consumer 在 ID 级检测到冒险后停顿，直到 producer 完成 WB 级（WB 前半段写、ID 后半段读，同周期可读），方可读到正确值。

每对停顿 = `max(0, WB(producer) − ID(consumer原始))`：

相关对	WB(producer) 周期	ID(consumer原始) 周期	停顿
I1→I2（R1）	5	3	5 − 3 = **2**
I2→I3（R3）	6	4	6 − 4 = **2**
I3→I4（R5）	7	5	7 − 5 = **2**
合计			**6**

（逐周期传播模拟确认：I1 WB=周期5，I2 WB=周期8，I3 WB=周期11，I4 WB=周期14，总停顿 = 14 − 8 = 6，与逐对求和一致。）

\boxed{总停顿周期 = 6,\quad 实际执行总周期 = 8 + 6 = 14}

（b）有转发机制

- **Load-use 相关对**（I1→I2，R1）：数据在 I1 的 MEM 级末尾才就绪，转发到 I2 的 EX 级仍需插入 **1 个停顿周期**。
- **ALU→ALU 相关对**（I2→I3、I3→I4）：通过 EX/MEM → EX 转发，**0 个停顿周期**。

相关对	类型	停顿
I1→I2（R1）	Load-use	**1**
I2→I3（R3）	ALU→ALU	**0**
I3→I4（R5）	ALU→ALU	**0**
合计		**1**

\boxed{总停顿周期 = 1,\quad 实际执行总周期 = 8 + 1 = 9}

结果汇总

项目	结果
时钟周期	500 ps
非流水线时间	6400 ps
理想流水线时间	4000 ps（8 周期）
加速比 S	1.6
吞吐率	1.00 指令/ns
无转发总停顿 / 实际周期	6 / 14
有转发总停顿 / 实际周期	1 / 9